



Технологии - 1

Работа с датой, временем, cookie и LocalStorage

1. Работа с датой и временем

На языке JavaScript переменные, которые хранят дату и время, называются переменными типа **date** ("дата"). Физически в компьютере такие переменные хранят число миллисекунд, прошедших с момента времени **00:00:00 1 января 1970 г.** (для дат до этого момента число миллисекунд отрицательное). Этот момент выбран не случайно. Он совпадает с моментом введения нового стандарта всемирного времени — стандарта UTC.

UTC (всемирное координированное время) было введено в 1970 г. вместо устаревшего **среднего времени по Гринвичу (GMT)**. Разница между ними менее секунды, поэтому в повседневной жизни, когда дробная часть секунд не важна, GMT рассматривается как эквивалент UTC.

Часовые пояса вокруг земного шара выражаются как положительное и отрицательное смещение от UTC (GMT) на конкретное количество часов.

Следует помнить, что время по UTC (GMT) не переводится ни зимой, ни летом. Поэтому для тех мест, где есть переход на летнее время, меняется и смещение относительно UTC (GMT).

В программе на языке JavaScript переменные типа "дата" создаются как копии встроеного в JavaScript объекта **Date()** командой вида:

имя-переменной = new Date();

Например, в случае исполнения команды

```
data = new Date();
```

переменная **data** получает при исполнении команды значения даты и времени (в миллисекундах), текущие в этот момент на компьютере.

Объект **Date()** имеет много разнообразных методов.

Метод **.toString()** переводит время в миллисекундах в строку времени GMT.

Например, в результате исполнения команды

```
d1 = data.toString();
```

переменная **d1** получает значение строки времени вида

Sat Mar 08 2014 23:58:11 GMT+0300 (Калининградское время (зима))

Это формат строки времени в браузере IE11. В других браузерах формат может несколько отличаться (меняется место данных года, не выводится географическое название).

Чтобы вывести только дату, используют метод **.toDateString()**.

Чтобы вывести только время, используют метод **.toTimeString()**.

Метод **.getTime()** позволяет получить то самое число миллисекунд, которое хранится в переменной типа "дата". Например, в результате исполнения команды

```
d2 = data.getTime();
```

переменная **d2** получает из переменной **data** хранимое там число миллисекунд.

Если в программе нужно задать какие-то конкретные время и дату, то используют команду вида

имя-переменной = new Date(гггг, мм, [дд, чч, мм, сс]);

где **гггг** — 4 цифры года (для XX века можно 1-2 цифры, например, 36 = 1936);

мм — число от 0 до 11 (месяц, 0 – январь);

дд — число от 1 до 31 (день месяца);

чч — число от 0 до 23 (часы в 24-часовом формате);

мм — число от 0 до 59 (минуты);

сс — число от 0 до 59 (секунды).

Например, команда

date1 = new Date(2003, 5, 8, 7, 25, 31);

задает переменной **date1** значение, которое соответствует 8 июня 2003 года 7 часов 25 мин 31 сек.

Скобки указывают на то, что аргументы (начиная с конца) можно не указывать. В этом случае они примут значения, начальные для данного вида аргумента (0 для всех, кроме дня месяца, для которого начальное значение 1).

А вот после года указывать в аргументах месяц обязательно, так как команда вида

имя-переменной = new Date(натуральное_число);

дает новой переменной значение числа миллисекунд с момента времени 00:00:00 1 января 1970 г.

Пример 35.

В условиях примера 20 по нажатию первой кнопки выведем в текстовое поле информацию о текущей дате и текущем времени, по нажатию второй кнопки — информацию о числе миллисекунд, а по нажатию третьей кнопки — информацию о времени и дате 11:23:34 7 октября 2013 года.

Естественно, в тексте файла **pos20.html** меняем номер примера, и файл сохраняем под именем **pos35.html**. Чтобы строка времени выводилась в текстовом поле полностью, в теге **<input>** увеличиваем размер текстового поля (**size**) до 80.

В теле первой функции **func1()** вместо команды, которая устанавливает цвет фона текстового поля, записываем команду, которая задает переменную типа "дата" с текущим значением времени:

```
var data = new Date();
```

Правую часть команды

```
document.getElementById("pp1").value="Вы нажали первую кнопку";
```

меняем на **data** и получаем

```
document.getElementById("pp1").value = data;
```

Заметим, что здесь мы не использовали метод **.toString()**. Браузеры автоматически применяют его в командах присваивания, когда переменная слева имеет тип "строка" (как в нашем случае).

В теле второй функции **func2()** вместо команды, которая устанавливает цвет фона текстового поля, опять записываем команду, которая задает переменную **data** типа "дата" с текущим значением времени:

```
var data = new Date();
```

Описание переменной **data** со служебным словом **var** в теле первой функции задало переменную, которой за пределами этой функции не существует. Поэтому в теле второй функции переменная **data** снова задается и используется.

Меняем вторую команду функции на

```
document.getElementById("pp1").value = data.getTime();
```

В теле третьей функции **func3()** вместо команды, которая устанавливает цвет фона текстового поля, записываем команду, которая задает переменную **data** типа "дата" с заданной информацией о времени:

```
var data = new Date(2013,9,7,11,23,34);
```

Меняем и вторую команду функции на

```
document.getElementById("pp1").value = data;
```

Сохраняем файл и запускаем его.

Можно заметить, что миллисекунд с 1 января 1970 г. накопилось не так и много, и каждое последующее нажатие на вторую кнопку естественно увеличивает число прошедших миллисекунд.

Методы объекта **Date** позволяют выделять элементы даты и времени, чтобы использовать их в программе.

Метод	Описание результата	Значения
<i>Определить дату и время, установленные в компьютере</i>		
getFullYear()	Год	Четырехзначные
getMonth()	Месяц	0–11
getDate()	Число месяца	1–31
getDay()	День недели	0–6 (0 – воскресенье, 1 – понедельник)
getHours()	Часы	0–23
getMinutes()	Минуты	0–59
getSeconds()	Секунды	0–59
getMilliseconds()	Миллисекунды (1/1000 сек)	0–999
<i>Определить дату и время по Гринвичу в соответствии с установленным в компьютере</i>		
getUTCFullYear()	Год	Четырехзначные
getUTCMonth()	Месяц	0–11
getUTCDate()	Число месяца	1–31
getUTCDay()	День недели	0–6 (0 – воскресенье, 1 – понедельник)
getUTCHours()	Часы	0–23

Все данные, приведенные во втором и третьем столбцах таблицы, можно задавать, используя методы с именами, которые начинаются со слова **set** вместо **get**.

Пример 36.

В условиях примера 35 по нажатию первой кнопки выведем в текстовое поле информацию о текущем часе, по нажатию второй кнопки — о текущем часе по

Гринвичу, а по нажатию третьей кнопки — изменим текущий час на 23 и выведем информацию об измененном времени и дате.

Меняем заголовок в тексте файла **pos35.html** на "Пример 36" и сохраняем его под именем **pos36.html**.

В теле первой функции **func1()** окончание второй команды **data** дополняем до **data.getHours()**.

В теле второй функции **func2()** окончание второй команды **data.getTime()** меняем на **data.getUTCHours()**.

В теле третьей функции **func3()** в первой команде удаляем аргументы, задающие конкретную дату, и добавляем после нее команду

```
data.setHours(23);
```

Сохраняем файл и запускаем его.

Сдвиг времени и даты на определенный промежуток, вычисление промежутка между двумя датами в программах на языке JavaScript проводится изменением числа миллисекунд.

Это число выводится методом **getTime()**, а устанавливается методом **setTime()**.

Пример 37.

В условиях примера 36 по нажатию первой кнопки выведем в текстовое поле информацию о текущем времени и дате, по нажатию второй кнопки текстовое поле должно очищаться, а по нажатию третьей кнопки — целое число, которое введено в текстовое поле, должно стать числом дней, которые добавляются к текущей дате, и новые данные о времени выводятся в текстовое поле.

Меняем заголовок в тексте файла **pos36.html** на "Пример 37" и сохраняем его под именем **pos37.html**.

В теле первой функции **func1()** окончание второй команды **data.getHours()** сокращаем до **data**.

В теле второй функции **func2()** удаляем первую команду, а окончание второй команды **data.getUTCHours()** меняем на **" "**.

В теле третьей функции **func3()** вместо второй команды

```
data.setHours(23);
```

записываем несколько команд.

Сначала вводим новую переменную **tn** и записываем ее значением содержимое текстового поля

```
var tn=document.getElementById("pp1").value;
```

Переводим строковое значение переменной **tn** в числовое

```
tn=parseInt(tn);
```

Так как в сутках $1000*60*60*24$ миллисекунд, то **tn** дней (суток) равно **tn*1000*60*60*24** миллисекунд. К текущему числу миллисекунд **data.getTime()** добавляем эти миллисекунды и результат записываем в переменную **data**

```
data.setTime(data.getTime() + tn*1000*60*60*24);
```

Сохраняем файл и запускаем его. Заметим, что задавать можно и отрицательный сдвиг дней.

Пример 38.

В окне браузера три текстовых поля (одно под другим) и еще ниже две кнопки **Вычислить** и **Очистить**. В первые два поля с клавиатуры цифрами вводятся даты в формате "дд мм гггг". По нажатию кнопки **Вычислить** в первых двух полях даты преобразуются в стандартные форматы даты в браузере, а в третьем поле появляется число дней, разделяющих эти даты.

Будем строить решение на базе файла примера 37. Меняем номер примера в заголовке текста на 38 и сохраняем файл под именем **pos38.html**.

В разделе **<body></body>** после тега

```
<input id="pp1" type="text" size=80 value="">
```

дописываем в текст еще два таких же тега, но с идентификаторами **pp2** и **pp3**.

После первого и второго тегов этих полей дописываем еще теги **
**.

В теге первой кнопки меняем атрибут

```
value = "Первая"
```

на

```
value = "Вычислить"
```

В теге второй кнопки аналогичный атрибут меняем на **"Очистить"**. А вот тег третьей кнопки удаляем.

Теперь работаем с функцией **func1()**. В начале тела этой функции вводим несколько пустых строк.

Записываем команду, которая записывает значение первого текстового поля как значение переменной **text**

```
var text=document.getElementById("pp1").value;
```

Потом значения переменной **text** разбиваем на отдельные значения, используя метод **split()** с разделителем " " (пробел):

```
var date=text.split(" ")[0];  
var mon =text.split(" ")[1]-1;  
var year=text.split(" ")[2];
```

Значение месяца сразу корректируем, так как в JavaScript месяцы нумеруются, начиная с 0.

Теперь корректируем две команды, которые остались тут из прошлого примера.

Чтобы задать переменную **dat1** типа "дата" с данными первого поля, команду

```
var data = new Date();
```

изменяем на

```
var dat1 = new Date(year,mon,date);
```

Чтобы вывести данные в новом формате, в команде

```
document.getElementById("pp1").value = data;
```

окончание **data** изменяем на **dat1.toString()**.

Для обработки первого текстового поля записаны 6 команд. Копируем их и вставляем ниже. А теперь изменим, адаптируя для обработки второго текстового поля.

В первой строке удаляем служебное слово **var** и меняем идентификатор **"pp1"** на **"pp2"**.

В строках 2–4 удаляем служебное слово **var**.

В пятой и шестой строках имя переменной **dat1** меняем на **dat2**. В шестой еще меняем идентификатор **"pp1"** на **"pp2"**.

Теперь запишем команды вычисления разницы в датах. Разницу в миллисекундах записываем как значение переменной **nn**.

```
var nn=Math.abs(dat1.getTime()-dat2.getTime());
```

Вычисляем теперь число дней (суток) разницы с округлением

```
nn=Math.floor(nn/(1000*60*60*24));
```

Остался вывод результата в третье текстовое поле

```
document.getElementById("pp3").value = nn;
```

Теперь работаем с функцией **func2()**.

Единственную команду в теле функции копированием повторяем еще два раза.

В копиях меняем идентификаторы на **"pp2"** и **"pp3"** соответственно.

Функцию **func3()** удаляем целиком. Сохраняем файл и запускаем его.

Пример 39.

Выведем в отдельном прямоугольнике цифровые (текстовые) показания текущего времени в формате "часы : минуты". Разделительное двоеточие должно раз в полсекунды исчезать и появляться.

Изменим текст файла **pos7.html**. Меняем номер примера и сохраняем файл с именем **pos39.html**.

Очищаем разделы **<script></script>** и **<body></body>**.

В разделе **<body></body>** записываем пару тегов **<div></div>** для организации вывода цифр. Между тегами запишем для примера текст: **23:45**.

Запускаем на исполнение. Отображение получилось довольно скромное. Изменим его, задавая в первом теге **<div>** стилевые свойства для атрибута **style**.

Увеличим размер цифр (**"font-size:28px;"**), зададим шрифт по-строже (**"font-family:Arial;"**), зададим для цифр белый фон (**"background:white;"**).

Теперь зададим размеры прямоугольника: высоту 30px (чуть больше, чем высота шрифта), ширину 150px (потом подберем минимально возможную). Задаем двойную черную границу шириной 7px (**"border: 7px double black;"**). Внутри прямоугольника для показаний задаем выравнивание «по середине» (**"text-align:center;"**).

После запуска проверяем ширину прямоугольника с часами и подбираем для нее минимально возможное значение.

Теперь организуем вывод текущего времени.

Даем тегу **<div>** идентификатор

```
id="cl"
```

В разделе **<script></script>** записываем команду присваивания переменной **dd** символьного значения " ", которое будем использовать как разделительный символ между числом часов и числом минут.

```
var dd=":";
```

Вносим конструкцию новой функции пользователя (первый) вариант. Даем функции имя **clock**.

В тело функции записываем стандартный набор команд:

- 1) создать переменную **data** типа «дата»;
- 2) забрать из нее показания числа часов;
- 3) забрать из нее показания числа минут.

```
var data = new Date();  
var h=data.getHours();  
var min=data.getMinutes();
```

Далее присвоим тегу **<div>** новое текстовое содержимое (показания часов)

```
document.getElementById("cl").innerHTML = h + dd + min;
```

Чтобы двоеточие исчезало и появлялось, будем менять его значение на «пробел» и обратно:

```
if (dd==":") dd=" ";  
else dd=":";
```

Теперь в HTML-тексте надо организовать запуск функции **clock()** через каждые полсекунды (500 мсек). Для этого в первый тег **<body>** дописываем атрибут запуска:

```
onload = "setInterval('clock()',500);"
```

Запускаем файл примера на исполнение. Выявляется одна тонкость. Если показания часов или минут меньше 10, то 0 перед единственной цифрой не выводится (показания «укорачиваются»). Исправим эту неточность. Будем добавлять цифру "0" принудительно для чисел, меньше 10. Для этого в теле функции **clock()** перед командой присвоения тегу **<div>** нового текстового содержимого вставляем команды добавления символа "0" в значения переменных **h** и **min**:

```
if (parseInt(h)<10) h="0"+h;  
if (parseInt(min)<10) min="0"+min;
```

Сохраняем файл и запускаем его на исполнение.

(!!! Пример 39 после выполнения показать преподавателю!!! 21)

Пример 40.

Построим на экране стрелочные часы, которые показывают текущее время. Пусть часы будут с круглым прозрачным циферблатом, диаметром 120px, синими цифрами, минутной и часовой стрелками синего цвета и секундной стрелкой красного цвета.

Изменим текст файла **pos39.html**. Меняем номер примера и сохраняем файл с именем **pos40.html**. Очищаем разделы **<script></script>** и **<body></body>**.

Для организации часов нужно создать в разделе **<body></body>** 12 заготовок типа **<div></div>**, содержащих цифры от 1 до 12. Первоначальное положение этих заготовок значения не имеет, но свойство **"position:absolute;"** надо задавать обязательно, чтобы потом положение можно было поменять.

Первая заготовка должна иметь вид

```
<div id="digs1" style="position:absolute;width:19px;font-family:Arial Black;font-size:14px;color:blue; text-align:center;">1</div>
```

Введем ее в текст примера. Кавычки обязательно используем двойные.

Вторая заготовка должна иметь вид

```
<div id="digs2" style="position:absolute;width:19px;font-family:Arial Black;font-size:14px;color:blue;text-align:center;">2</div>
```

и т.д. до 12. Но вводить в текст пока надо только первую заготовку, так как выписать 12 тегов один за другим, конечно, можно, но с помощью цикла на языке JavaScript сделаем запись короче.

Для этого в разделе **<body></body>** вводим перед первым тегом **<div>** еще один раздел для кода JavaScript:

```
<script>  
  
</script>
```

и записываем в него конструкцию повторения

```
for (i=1;i<13;i++) {  
  
}
```

Теперь в этой конструкции теги в текст будем выводить командой **document.write()** с разными цифрами для циферблата и для идентификаторов. Команда будет иметь вид

```
document.write('<div id="digs'+i+' " style="position:absolute;  
width:19px; font-family:Arial Black; font-size:14px; color:blue;  
text-align:center;">'+i+'</div>');
```

Чтобы ввести эту команду, сначала записываем

```
document.write();
```

Добавляем как аргумент пару одинарных кавычек

```
document.write('');
```

А потом копируем и выводим между кавычками первую заготовку для циферблата из текста примера. Осталось внести изменения для имени идентификатора и задаваемой цифры: в двух местах вместо цифры "1" надо вписать код **'+i+'**.

Исполнение кода JavaScript создаст несколько тегов HTML. Такой HTML-текст называется "живым" HTML.

Теперь записанную сначала заготовку для циферблата можно удалить.

Стрелки для часов будем образовывать прямыми цепочками из одноцветных рисунков: **darkblue.jpg** (темно-синий) — для часовой и минутной стрелок, **red.jpg** (красный) — для секундной стрелки. Эти два рисунка размерами 1×1px нужно заготовить с помощью Paint и поместить их в рабочую папку.

Диаметр циферблата 120px, радиус — 60px, значит для минутной и секундной стрелок нужно не менее 60 рисунков. Для часовой стрелки используем 35 рисунков (она короче). Тут уже без цикла точно не обойтись.

В часовой стрелке будем использовать цепочку из рисунков размером 3×3px. В цикле будем менять и номера идентификаторов. Запишем цикл в текст веб-страницы в том же разделе JavaScript.

```
for (i=1;i<36;i++) {
```



```
document.write('');  
}
```

Для минутной стрелки будем использовать цепочку рисунков размером 3×3px. Дописываем конструкцию

```
for (i=1;i<61;i++) {  
document.write('');  
}
```

Для секундной стрелки используем файл **red.jpg** с рисунком размером 1×1px (оставляем исходный размер). Дописываем:

```
for (i=1;i<61;i++) {  
document.write('');  
}
```

Для минутных и секундных стрелок используем две разные конструкции повторения, хотя для них число составляющих элементов одно и то же (60). Дело в том, что секундная стрелка должна быть видна поверх минутной. Поэтому ее элементы должны выводиться на экран после того, как выведены все элементы минутной стрелки.

Заготовки для элементов стрелочных часов готовы, но не задано их положение.

Выбираем центром циферблата координаты точки экрана и дописываем их после конструкций повторения

```
Xc=500;  
Yc=400;
```

Положение секундной и минутной стрелок будем менять на угол, равный 1/60 от 360° (60 минут в часе). В радианах — это $2\pi/60$. Зададим и допишем константу

```
K= Math.PI/30;
```

Располагаем цифры по окружности радиуса 60px с углом 1/12 окружности между ними, что равно $5 \cdot K$.

```
for (i=1;i<13;i++){  
document.getElementById("digs"+i).style.top = +  
Yc - 8 - 60*Math.cos(5*K*i);  
document.getElementById("digs"+i).style.left = +  
Xc - 8 + 60*Math.sin(5*K*i);  
}
```

Вопрос: Зачем нужны сдвиги $- 8$?

Сохраняем файл и запускаем его на исполнение. Должны отобразиться цифры на циферблате. При необходимости можно изменить положение его центра.

Теперь в первый пока пустой раздел JavaScript дописываем конструкцию функции пользователя для новой функции **clock()**, в которой будем рассчитывать положение стрелок.

В тело функции **clock()** сначала записываем стандартный набор команд:

1) Создать переменную **data** типа «дата».

- 2) Забрать из нее показания для часов;
- 3) Забрать из нее показания для минут.
- 4) Забрать из нее показания для секунд.

```
var data = new Date ();
var sec = data.getSeconds();
var min = data.getMinutes();
var h = data.getHours();
```

Далее записываем конструкцию повторения

```
for (i=1;i<61;i++){
}
```

а уже в нее — команды присвоения отдельным картинкам вычисленного текущего положения (сначала для секундной и минутной стрелок):

```
document.getElementById("secs"+i).style.top = 
Yc + 1 - (i-1)*Math.cos(K*sec);
document.getElementById("secs"+i).style.left = 
Xc + 1 + (i-1)*Math.sin(K*sec);
document.getElementById("mins"+i).style.top = 
Yc - (i-1)*Math.cos(K*min);
document.getElementById("mins"+i).style.left = 
Xc + (i-1)*Math.sin(K*min);
```

Часовая стрелка короче, поэтому тут же в теле цикла дописываем

```
if (i<36){
    document.getElementById("hrs"+i).style.top = 
    Yc - (i-1)*Math.cos(5*K*h + K*min/12);
    document.getElementById("hrs"+i).style.left = 
    Xc + (i-1)*Math.sin(5*K*h + K*min/12);
}
```

А вот команду запуска циклического повторения функции **clock()** дописывать не нужно. Она осталась из предыдущего примера. Для выделения циферблата сделаем фон окна светлее: в первом теге **<body>** меняем цвет фона с **green** на **lightgreen**.

Сохраняем файл и запускаем его на исполнение.

Задания.

1). Изменить текст примера 37 так, чтобы по нажатию третьей кнопки — целое число, которое введено в текстовое поле, должно стать числом часов, которые добавляются к текущему времени, и новые данные о времени выводятся в текстовое поле.

2) Изменить текст примера 38 так, чтобы в первые два поля с клавиатуры цифрами вводились моменты времени в формате "чч:мм:сс". По нажатию кнопки **Вычислить** в первых двух полях время преобразовалось в стандартные форматы времени в браузере, а в третьем поле появляется число минут, разделяющих эти моменты времени.

3). Изменить текст примера 39 так, чтобы часы отображали еще и секунды.

4) Используя конструкции для перетаскивания, изменить текст примера 39 так, чтобы построенные цифровые часы можно было перетаскивать по окну браузера.

5). Используя конструкции для перетаскивания, изменить текст примера 40 так, чтобы построенные стрелочные часы можно было перетаскивать по окну браузера.

(!!! Задание 5 после выполнения показать преподавателю!!! 22)

6). Создать веб-страницу с прямоугольным блоком, в котором отображается календарь на текущий месяц и выделен фон числа текущего дня.

2. Работа с cookie

Технология **cookie** ("ку́ки", в переводе с англ. — "печенье") — это технология хранения данных, которые связаны с веб-страницами и называются **cookie**. Технология позволяет хранить данные небольшого объема (до 4 кБ).

Работать с cookie позволяют программы на языке JavaScript.

В компьютере cookie хранятся в виде отдельных записей в специальных файлах, если такая возможность не выключена настройкой браузера.

Cookie записываются, считываются и удаляются браузерами по командам, которые поступают от веб-серверов или от пользователя.

При первом обращении к сайту браузер по команде веб-сервера делает соответствующую запись cookie. При любой попытке повторно открыть веб-страницу этого сайта браузер пересылает веб-серверу эту запись cookie в виде HTTP-запроса.

Cookie обычно используются для:

- аутентификации пользователя;
- хранения персональных предпочтений и настроек пользователя, включая коды доступа на сайт;
- отслеживания предпочтений пользователя;
- ведения статистики о пользователях.

Для одной веб-страницы браузер может хранить несколько разных записей cookie. В файле браузер записывает их вместе в единую текстовую строку.

Каждая запись cookie получает имя, поэтому в текстовой строке разные записи браузер различает по именам.

Запись cookie состоит из нескольких разделов, часть из которых обязательные. Разделы отделяются друг от друга точками с запятой.

Каждый раздел записывается в форме равенства

имя_атрибута = значение

Первый раздел (обязательный) записи cookie — это, понятно, раздел, в котором есть имя записи. Он имеет вид:

имя_записи=данные

где **имя_записи** — слово без пробелов, записанное по правилам записи имен переменных языка JavaScript;

данные — строка символов, которая не должна содержать пробелов, точек с запятой и запятых.

Если в сохраняемых данных есть недопустимые знаки, то их **заменяют особыми кодами**. Например, символ побела заменяется кодом «%20».

В JavaScript такую замену автоматизирует функция **escape()**. Аргументом функции записывают обычную текстовую строку, а результатом будет текстовая строка, в которой недопустимые для cookie символы заменены кодами. Например, результатом исполнения функции

```
escape("Виктор Кузьмин")
```

будет строка

```
"Виктор%20Кузьмин".
```

Пусть у нас первый раздел записан в виде

```
frend=Виктор%20Кузьмин
```

Имя записи **frend**, данные — **Виктор Кузьмин**.

Второй раздел (обязательный) имеет вид:

```
expires=строка_времени_GMT
```

expires — служебное имя раздела, справа — дата и время окончания срока хранения.

Например:

```
expires=Mon, 01-Oct-14 00:00:01 GMT
```

Если дата и время окончания срока хранения cookie не заданы или просрочены, то при завершении сеанса работы браузер такую запись удаляет.

Третий раздел (необязательный):

```
path=путь
```

path — служебное имя раздела, справа указывается **ПУТЬ** к другой папке (не к той, что по умолчанию), в которой будут храниться копии cookie.

Четвертый раздел (необязательный):

```
domain=имя_домена
```

domain — служебное имя раздела, справа доменное имя (не менее двух меток), которое записывает браузер, чтобы была возможность просмотреть все cookie, относящиеся к этому домену.

Пятый раздел (необязательный) имеет вид:

```
secure=логическое_значение
```

secure — служебное имя раздела, справа константа **true** или **false**.

В разделе определяется необходимость безопасного интернет-соединения с доменом.

Если исключить необязательные разделы, то в нашем случае получается запись cookie с двумя разделами:

```
"frend=Виктор%20Кузьмин; expires=Sat, 01-Oct-16 00:00:01 GMT"
```

Внимание! Разбираемые далее примеры для работы с cookie работают в браузерах **IE, Mozilla Firefox и Safari**.

Браузеры **Google Chrome** и **Opera** локальным файлам работать с cookie не позволяют, но в файлах, загруженных из интернета, все работает и в этих браузерах.

Все cookie, которые связаны с конкретной веб-страницей, хранит объект DOM с именем **document.cookie**.

Чтобы создать запись cookie, связанную с веб-страницей, нужно программой JavaScript, встроенной в эту веб-страницу, присвоить объекту **document.cookie** строковое значение, например:

```
document.cookie="frend=Виктор%20Кузьмин; expires=Sat, 01-Oct-16 00:00:01 GMT";
```

Запись и чтение cookie часто организуют специальными функциями JavaScript.

Функция **writeCookie()** для записи cookie имеет вид:

```
function writeCookie(name,value,ds) {  
var date= new Date();  
date.setTime(date.getTime() + ds*1000*60*60*24);  
document.cookie=name + "=" + escape(value)+ "; expires=" +  
date.toGMTString();  
}
```

Аргументы функции **writeCookie()**:

name — переменная для имени создаваемой записи;

value — переменная для данных записи;

ds — переменная для записи числа дней хранения записи, начиная с текущей даты. Если задать **ds=-1**, то устанавливается прошедшая дата, и запись браузером удаляется.

Функция **readCookie()** для чтения cookie имеет вид:

```
function readCookie(name) {  
var cookie = document.cookie;  
var search = name + "=";  
var str = underfined;  
var offset = 0;  
var end = 0;  
if (cookie.length > 0) {  
    offset = cookie.indexOf(search);  
    if (offset != -1) {  
        offset += search.length;  
        end = cookie.indexOf(";", offset)  
        if (end == -1) {  
            end = cookie.length;  
        }  
        str = unescape(cookie.substring(offset, end));  
    }  
}  
return str;  
}
```

Аргумент **name** функции **readCookie()** предназначен для ввода конкретного имени записи.

Значением этой функции будут **данные**, хранимые в cookie с именем, которое записано как значение **name**. Если записи с заданным именем нет, то выведется значение **underfined** (не определено).

Визуальная сложность функции **readCookie()** связана с тем, что текстовая строка **document.cookie** хранит все cookie, связанные с веб-страницей, и в этой строке сначала надо найти конкретную запись по ее имени.

Пример 41.

В условиях примера 36 используем текстовое поле формы для ввода и вывода **данных** из cookie. Первую кнопку с надписью "Очистить" будем использовать для очистки поля, вторую "Записать" — для записи информации из текстового поля в cookie, третью "Вывести" — для вывода данных из cookie по ее имени. Для создания cookie в текстовое поле будем вводить **имя записи, данные и число дней хранения**, разделяя их точкой с запятой. Для считывания информации из cookie в текстовое поле будем вводить только **имя записи**.

Меняем номер примера в тексте файла **pos36.html** и сохраняем его под именем **pos41.html**.

Меняем надписи на кнопках.

В раздел для JavaScript дописываем функции **writeCookie()** и **readCookie()**, которые были рассмотрены ранее.

Далее в теле первой функции **func1()** оставляем только команду

```
document.getElementById("pp1").value = "";
```

Переходим ко второй функции **func2()**. В теле этой функции значение текстового поля записываем как значение переменной **text**

```
text= document.getElementById("pp1").value;
```

Потом значения переменной **text** разбиваем на отдельные значения, используя метод **.split()** с разделителем **";"**:

```
name=text.split(";")[0];  
value=text.split(";")[1];  
ds=text.split(";")[2];
```

и вызываем функцию записи

```
writeCookie(name,value,ds);
```

Напоследок очищаем текстовое поле

```
document.getElementById("pp1").value = "";
```

В теле третьей функции **func3()** значение текстового поля записываем как значение переменной **name**

```
name=document.getElementById("pp1").value;
```

и выводим содержание записи cookie с именем, заданным в **name**, в текстовое окно в форме

имя_записи=данные

Вот эта команда

```
document.getElementById("pp1").value= name+'='+readCookie(name);
```

Сохраняем файл и запускаем его на исполнение. Далее проверим работу программы:

- 1) создаем запись cookie — вводим в текстовое поле информацию по форме «**имя_записи; данные; число дней хранения**» и нажимаем кнопку "Записать";
- 2) очищаем текстовое поле кнопкой "Очистить";

3) выводим имя и данные записи cookie — вводим имя записи cookie в текстовое поле и нажимаем кнопку "Вывести".

(!!! Пример 41 после выполнения показать преподавателю!!! 23)
--

Задания.

1. Создайте форму с четырьмя кнопками, в которой четвертая кнопка "Удалить" позволяет удалить cookie по ее имени (запись cookie удаляется, когда дата окончания срока ее хранения становится прошедшей датой).

2. В условиях задания 1 добавьте пятую кнопку, которая позволяет выводить в диалоговое окно все cookie, связанные с веб-страницей (все cookie хранит строка **document.cookie**).



3. Работа с LocalStorage

Технология LocalStorage — это еще одна технология хранения данных, связанных с веб-страницами. Данные хранятся в специальном файле, который называют **LocalStorage** (**локальное хранилище** или **HTML5-хранилище**).

Технология LocalStorage является развитием технологии cookie и позволяет хранить записи объемом 5-10 Мб.

Данные в LocalStorage записываются, считываются и удаляются браузерами по командам, которые поступают от веб-серверов или от пользователя.

Отдельная запись в LocalStorage является текстовой строкой, которая имеет форму равенства

имя_записи=данные

Работать с LocalStorage позволяют программы на языке JavaScript.

Внимание! Разбираемые далее примеры для работы с LocalStorage работают в браузерах **Google Chrome, Opera, Mozilla Firefox и Safari**. Браузер **IE** локальным файлам работать с LocalStorage не позволяет, но в файлах, загруженных из интернета, все работает.

Локальное хранилище в JavaScript является объектом BOM с именем **window.localStorage** или коротко **localStorage**. Объект **localStorage** языка JavaScript имеет несколько методов:

localStorage.setItem(n,d) — сохранение записи с именем **n** и данными **d**;

localStorage.getItem(n) — считывание данных записи с именем **n**;

localStorage.removeItem(n) — удаление записи с именем **n**;

localStorage.clear() — очистка локального хранилища.

Например, чтобы записать под именем **"frend"** данные **"Виктор Кузьмин"** в LocalStorage достаточно записать команду

```
localStorage.setItem("frend", "Виктор Кузьмин");
```

Чтобы сделать значением переменной **dat** данные этой записи из LocalStorage, используют команду

```
dat=localStorage.getItem("frend");
```

Пример 42.

В условиях примера 41 используем форму для ввода и вывода **данных** из LocalStorage. Оставим назначение кнопок и форму вывода данных, заданные в примере 41. Для создания записи в LocalStorage в текстовое поле будем вводить имя_записи и данные, разделяя их знаком "=" (равно). Для считывания данных записи оставим форму ввода из примера 41 (только имя записи).

Меняем номер примера в тексте файла **pos41.html** и сохраняем его под именем **pos42.html**.

Удаляем функции **writeCookie()** и **readCookie()** из раздела для JavaScript.

В функции **func2()** (функция для записи):

- во второй и третьей командах меняем разделяющий знак ";" на "=",
- четвертую команду удаляем,

– команду

```
writeCookie(name,value,ds);
```

меняем на команду

```
localStorage.setItem(name,value);
```

В функции **func3()** (функция для чтения) во второй команде меняем функцию

```
readCookie(name)
```

на

```
localStorage.getItem(name)
```

Сохраняем файл и запускаем его на исполнение. Далее создаем запись (вводим информацию в текстовое поле и нажимаем вторую кнопку), очищаем текстовое поле (первая кнопка), вводим имя записи и выводим ее имя и данные в текстовое поле (третья кнопка).

(!!! Пример 42 после выполнения показать преподавателю!!! 24)

Задания.

1. Создайте форму с пятью кнопками, в которой четвертая кнопка "Удалить" удаляет из хранилища запись по ее имени, а пятая кнопка "Удалить все" очищает хранилище.

2. В программе предыдущего задания добавьте в форму шестую кнопку "Сумма" со следующей функцией: если данные записи являются набором чисел через запятую, то в текстовое окно выводятся данные по форме для кнопки "Вывести", которые завершаются знаком "=" и числом-суммой чисел из набора. Например,

```
mm=2,3,-1,11,-4=11
```